Lista de Exercícios de Python usando paradigma Funcional

Nesta lista, vocês devem usar o paradigma funcional. Isto é: você deve usar apenas funções sem efeitos colaterais; **iterações devem ser feitas apenas usando recursão**. Não usem os métodos prontos de Python (a não ser quando explicitado)!

Lembrem-se: Funções, em programação funcional pura, seguem o padrão das funções matemáticas. Isso é, elas **não são sequências de comandos**, ou elas são uma expressão ou uma função definida por várias partes (cada parte pode ter apenas uma expressão). Ex:

```
def exemplo(x, y):
    if (x>0):
        if (y>0):
            return x*y
        else:
            return 0
    else:
        return 0
```

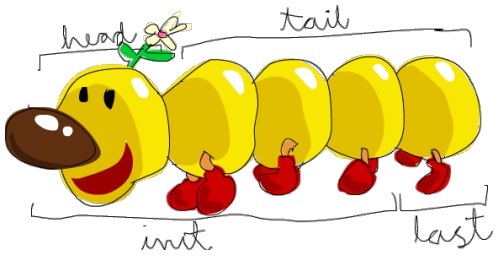
```
def exemplo(x, y):
    if (x>0) and (y>0):
        return x*y
    if (x>0) and not (y>0):
        return 0
    if not (x>0):
        return 0
```

Questão 1) Escreva uma função 'head' que retorna o primeiro elemento de uma lista

Questão 2) Escreva uma função 'tail' que retorna toda a lista, exceto o primeiro elemento

Questão 3) Escreva uma função 'init' que retorna toda a lista, exceto o último elemento

Questão 4) Escreva uma função 'last' que retorna o último elemento de uma lista



A partir daqui, pare de usar slices! Use apenas as 4 funções acima para acessar as listas

Questão 5) A sequência de Fibonacci é dada pela seguinte série: 0 1 1 2 3 5 8 13 ... Em termos matemáticos, a sequência de Fibonacci pode ser definida através da seguinte relação de recorrência:

$$f(n) = \begin{cases} 0, & n=0 \\ 1, & n=1 \\ f(n-1) + f(n-2) \end{cases}$$

Construa uma função para retornar o n-ésimo termo da sequência.

Questão 6) Escreva uma função que recebe um número N e calcula o seu fatorial: N!

Questão 7) Escrever um programa que lê um valor N inteiro e positivo e que calcula e escreve o valor de E = 1 + 1 / 1! + 1 / 2! + 1 / 3! + … + 1 / N!

Questão 8) Faça um algoritmo que leia vários números inteiros e calcule o somatório apenas dos números negativos digitados. O fim da leitura será indicado pelo número 0.

Questão 9) Escrever um programa que lê N números reais, um de cada vez, e conta quantos destes valores são negativos, escrevendo esta informação.

Questão 10) Escreva uma função que retorne o tamanho (a quantidade de elementos) de uma lista. Não usar a função len para isso.

Questão 11) Escreva a função ehPrimo para verificar se um número dado é primo.

Questão 12) Faça um algoritmo que calcule os N primeiros números primos

Questão 13) Faça uma função que concatena duas listas de forma recursiva. Utilize as funções head/tail para acessar os elementos. O comportamento deve ser o mesmo do operador + (listas). O operador + até pode ser usado, mas um dos operandos deve conter no máximo 1 elemento.

Questão 14) Escreva uma função que verifique se um elemento pertence a uma lista. Não usar o operador “in”;

Questão 15) Escreva uma função para realizar a união (de conjuntos matemáticos) de duas listas. A função é similar à feita na Q13, mas elementos repetidos não são permitidos.

Questão 16) Defina uma função que dada uma lista de inteiros e um número n, retorne o total de elementos de valor superior a n.

Questão 17) Defina uma função que dada uma lista de inteiros e um número n, retorne outra lista contendo apenas de elementos de valor superior a n. Use a função feita na Q13.

Questão 18) Escreva uma função que inverte o conteúdo de uma lista. Use apenas as funções da Q1 e a da Q13:

invertelista (“abcd”) = “dcba”.

Questão 19) Escreva uma função que receba uma palavra e gere seu palíndromo. Ex.:

geraPalindromo (“abcd”) = “abcddcba”.

Questão 20) Defina a função strip que dadas duas listas, retira da segunda todos os elementos que ocorrem na primeira, em qualquer quantidade.

Questão 21) Defina a função consoantList que recebe dois parâmetros: o primeiro é uma string que contém apenas consoantes, e o segundo é uma string “livre”. A função retorna verdadeiro se somente se todas as consoantes da segunda lista, incluindo repetições, ocorrem na primeira lista, na mesma ordem. Exemplos:

consoantList ("sdd", "saude") -> False

consoantList ("sdd", "saudade") -> True

Dica: use a função strip.